Bengaluru Flood-Risk Map using MPI

Running hundreds of rainstorms over the real terrain of Bengaluru to find the places that flood most often, with the storms shared across many processes

Course	Parallel Computing (PCAP) - Project Synopsis
Programme	B.Tech, Computer Science and Engineering
Made by	Ritwik - 245805240 Atharv - 245805386 Pranshu - 245805158
Tools	C with OpenMPI; Python (NumPy, Matplotlib) for the maps and charts; raylib for the live demo

1. Summary

Bengaluru floods badly in heavy rain: roads such as the Outer Ring Road and junctions such as Silk Board turn into lakes almost every monsoon. Water runs downhill and collects in low ground, so the shape of the land alone already says a lot about where the water will go.

This project takes a real elevation map of Bengaluru from satellite data and lets hundreds of different storms fall on it, from short light showers to long heavy downpours. A deliberately simple water model moves the rain downhill, step by step. For every patch of ground the program counts in how many of the storms it ends up under water. The result is a colour-coded flood-risk map of the city, a ranked list of the most flood-prone areas, and a check against the places the city corporation (BBMP) has officially listed as flood-prone.

The aim is to learn parallel programming, not hydrology. The storms do not depend on each other, so they can be shared out among many processes. Everything to do with dividing the work, passing messages between processes and combining the results is written by us with MPI. The map is a screening map, not a forecast.

2. What the program has to do

Step	In plain words
1. Load the terrain	Read the height of the ground for every cell of the map: 1,116 x 1,188 cells of about 30 m, covering about 1,200 km^2 of Bengaluru (FABDEM satellite data, with buildings and trees removed); about 990 km of mapped drains and streams are cut into it
2. Make the storm list	Build 400 storms: 20 rain intensities (10 to 60 mm per hour) times 20 durations (1 to 4 hours)
3. Run each storm	Rain falls on every cell; every 30 simulated seconds each wet cell passes some of its water to its lower neighbours; after the rain stops the water drains for 2 more hours. A cell counts as flooded if it is still more than 0.3 m deep
4. Combine the storms	For every cell, count in how many storms it flooded and remember the deepest water it ever had
5. Report	Draw the flood-risk map, rank 500 m areas by risk, compare the ranking with 386 official BBMP flood points, and tabulate the flooded area per storm

Table 1: The five steps of the flood-risk program.

3. Where the parallel work is

Not every step splits equally well, and that is what makes this project worth studying.

- **Step 3 is almost all of the time, and it splits perfectly by storm.** One storm on the 30 m map takes between about 4 and 14 seconds on one core, so 400 storms take about an hour on one process. The storms are completely independent: every process keeps its own copy of the map and runs whole storms without talking to anyone.
- **But the storms are not equal.** A 4-hour storm needs twice as many time steps as a 1-hour storm, and a wet map takes longer to compute than a dry one. If each process is simply given a fixed share of the storm list, some finish long before others and sit idle.
- **Step 4 is a combining step.** The flood counts from all processes must be added together cell by cell, and the deepest water kept. This happens once at the end, and its cost does not shrink when more processes are added.
- **Steps 1, 2 and 5 are small** and run on one process (step 5 in Python, after the run).

The limits worth reporting are the uneven storm lengths, the final combining step, and the fact that the processes share one laptop's memory and cores, so each extra process helps a little less than the one before.

4. How the processes work together

4.1 Sharing the settings

The first process reads the command line (the map file, the rain ranges, the flood depth) and broadcasts the settings to every other process. Each process then loads the map from disk and builds the storm list itself, so no map data ever has to be sent between processes. Only storm numbers travel.

4.2 Handing out storms on demand (dynamic mode)

The first process becomes a coordinator that only hands out work and does not simulate. Each worker sends a "ready" message that also carries the result of its last storm, and gets back the number of the next storm, or a stop signal when none are left. A worker that drew a short storm simply comes back sooner, so everyone stays busy.

4.3 Fixed shares (static mode, for comparison)

Each worker is given one fixed, contiguous block of the storm list up front and runs it without any messages. This is simpler but cannot react to storms taking different times. Running both modes on the same storms shows how much the on-demand version gains.

4.4 Bringing the results together

At the end, the flood counts of all processes are added together and the deepest water is combined, cell by cell, at the first process. The busy time of every process is gathered too, so the program can report how evenly the work was shared. The first process then writes the result files.

4.5 Checking that the answer is right

The water model has no randomness, so the parallel program must give exactly the same answer as a plain one-process version. The output files of both are compared byte for byte for every mode and number of processes.

4.6 Showing it live

A separate window (raylib) replays one storm step by step on an OpenStreetMap street map, so the audience can watch the water spread, zoom into an area, and read the height and water depth of any point.

5. MPI features used

MPI feature	What it does, in plain words
Broadcast	The first process sends the same settings (map file, rain ranges) to everybody
Send and receive	Workers ask for the next storm and the coordinator answers; "receive from anyone" lets it serve whichever worker is ready first
Reduce	Adds up the flood counts of all processes cell by cell (sum), and keeps the deepest water seen (maximum)
Gather	Collects every process's busy time, and in static mode the per-storm results, at the first process
Barrier	Makes all processes start and stop the clock together
Timer	Measures the total run time and each process's busy time

Table 2: The MPI ideas the project covers.

6. What we found

6.1 The parallel answer is exactly right

Dynamic and static mode on 2 and 4 processes produce result files that are byte-for-byte identical to the one-process program (checked with *cmp*, and re-checked after every change to the model). Water is conserved to within 0.0003%.

6.2 Speed

Benchmark on the 90 m version of the map (400 storms, 3 repetitions, median wall time, laptop on mains power):

Processes	Dynamic	Static	Speed-up (dynamic / static)	Load imbalance (dynamic / static)
1	402 s	-	1.0x	-
4	246 s	253 s	1.6x / 1.6x	1.00 / 1.04
8	170 s	178 s	2.4x / 2.3x	1.01 / 1.05
14	131 s	138 s	**3.1x / 2.9x**	**1.02 / 1.10**

Table 3: Speed-up and load imbalance (busiest worker / average worker).

- **Handing out storms on demand balances the work.** Dynamic mode keeps the busiest worker within 2% of the average; fixed shares reach 8-10%. In the 3-repetition medians dynamic is also faster from 4 processes up, but only by about 5%, and a later single-repetition check (2.8x dynamic / 3.1x static at 14 processes) showed that run-to-run noise on this laptop (10-20%) is larger than that gap. The imbalance numbers are the clearer evidence.
- **The speed-up flattens because of the laptop, not MPI.** The same storm takes 1.0 s when it runs alone but 4.1 s when 13 run at the same time: all processes share one laptop's cache, memory bandwidth and power budget, and the CPU mixes fast and slower cores. MPI's own cost is tiny (each message is a few bytes).
- The full 400-storm run on the 30 m map takes about 14 minutes on 12 processes, against about an hour on one.

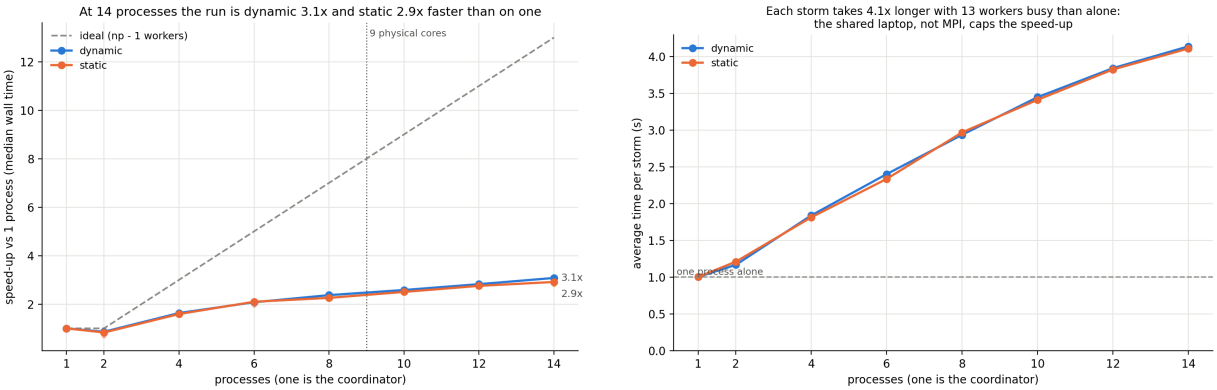


Figure 1: Left, speed-up against the ideal line. Right, the average time per storm grows as more processes share the laptop.

6.3 Where Bengaluru floods

- Flood risk follows the city's natural drainage: valleys, drain lines and chains of lakes. Clusters stand out in the west and centre (Vijayanagar, Malleshwaram, Rajajinagar) and around the south-east lakes (Bellandur, Outer Ring Road).
- The flooded area grows strongly with rain: 13.5 km^2 for the lightest storm (10 mm/h for 1 h), 58 km^2 for 60 mm/h for 1 h and 191 km^2 for the heaviest (60 mm/h for 4 h). Intensity matters more than duration.
- **87 of 384 official BBMP flood points lie in the riskiest 10% of the map's 500 m areas: 2.3 times more often than random places would.** 6 of the 10 highest-ranked areas have a BBMP flood point within 1 km.

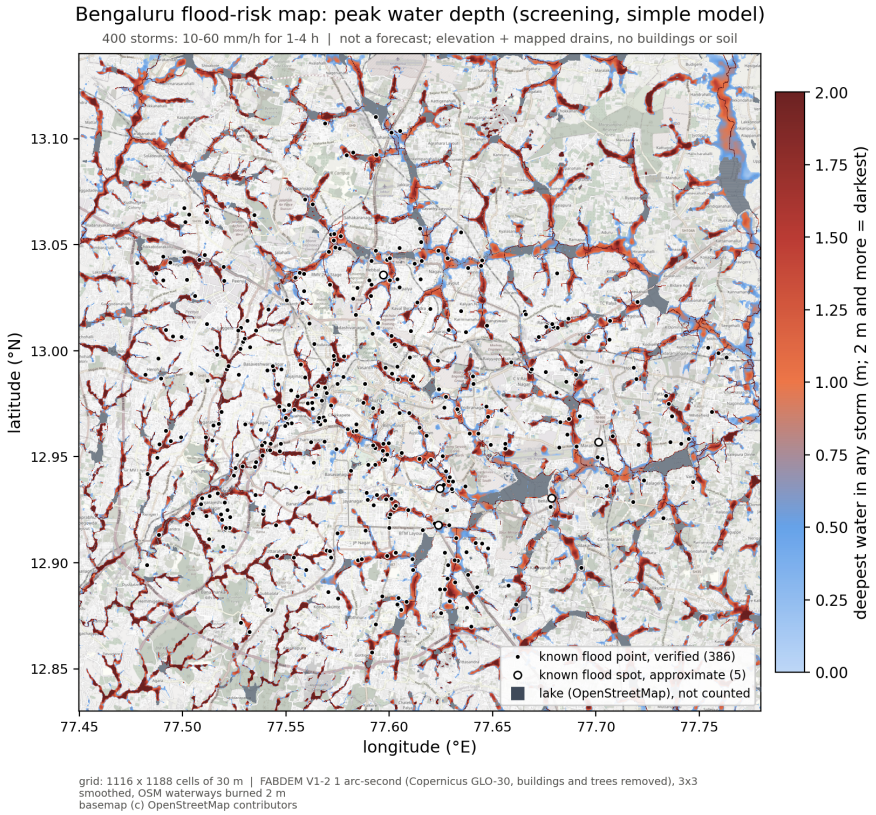


Figure 2: Deepest water reached in any of the 400 storms (30 m cells, mapped drains included), over an OpenStreetMap street map. Black dots: BBMP flood points; grey: lakes.

6.4 Making the map more accurate

Each improvement was tested on half of the BBMP points and scored on the other half, so the result is not tuned to the test. The score (AUC) is the chance that a real flood point is rated riskier than a random place: 0.5 means guessing, 1.0 perfect.

Version of the model	Score on unseen points (AUC, within 300 m)
Original terrain (SRTM, includes rooftops and trees)	0.55
Better terrain (FABDEM, buildings and trees removed)	0.68
+ about 990 km of mapped drains cut into the terrain	0.71
+ risk scored by the deepest water instead of water left at the end	0.71

Table 4: How each change improved agreement with real flood points.

Soaking rain into the ground (a constant loss rate) and a wider range of storms were also tested and did not improve the agreement.

6.5 Limits

This is a screening map, not a forecast. The model has no drain capacity or blockages, no soil, no buildings and no underpasses, so about 7 in 10 real flood points still fall outside the riskiest zone, and depths and times are model values. Timings on this laptop vary between sessions, so all speed comparisons come from one benchmark session.

7. Tools

Language	C (C11), compiled with the MPI compiler wrapper (mpicc)
MPI library	OpenMPI 4.1
Terrain data	FABDEM V1-2 (Copernicus 30 m elevation with buildings and trees removed, University of Bristol); SRTM 30 m for comparison
Flood points	386 BBMP flood-vulnerable, flood-prone and low-lying locations (open data from OpenCity, source KSRSAC)
Map data	OpenStreetMap: drains and streams (cut into the terrain), lakes (left out of the ranking) and the street map behind the results, (c) OpenStreetMap contributors
Live demo	raylib, a small C graphics library
Machine	A 14-core laptop (Intel Core Ultra 5 125H) running Ubuntu 24.04 under WSL; tested up to 14 processes
Graphs	Python with NumPy and Matplotlib, for the maps and timing charts in the report